

Para el ecosistema de Flutter, la alternativa reina es **flutter_map**. Este paquete es una implementación pura en Dart basada en los conceptos de **Leaflet** (la librería de mapas web más famosa del mundo). Funciona consumiendo imágenes de mapas fragmentadas llamadas *Tiles* (Mosaicos) directamente de servidores gratuitos como **OpenStreetMap**.

Cartografía Digital Open Source e Integración de Mapas con GPS

Este módulo enseña a integrar un entorno cartográfico interactivo dentro de Flutter sin dependencias comerciales, renderizando capas de mosaicos geospaciales y posicionando marcadores dinámicos alimentados por el hardware de geolocalización del dispositivo.

1. Fundamentos Teóricos y Conceptos de **flutter_map**

Para desarrollar sistemas de información geográfica (SIG) en móviles, se debe dominar la arquitectura de capas en la que se basan los mapas digitales:

- **MapController**: El objeto de control que permite manipular el mapa mediante código. Con él se puede cambiar el zoom, centrar la cámara en unas coordenadas específicas o mover el mapa programáticamente cuando el GPS detecte movimiento.
 - **TileLayer (Capa de Mosaicos)**: Un mapa digital del mundo no es una sola imagen masiva; está fragmentado en millones de imágenes cuadradas de 256×256 píxeles llamadas *Tiles*. Conforme el usuario arrastra el mapa o hace zoom, **flutter_map** calcula qué mosaicos necesita y los descarga de forma asíncrona mediante peticiones HTTP a los servidores de **OpenStreetMap**. URL base estándar: `[https://tile.openstreetmap.org/](https://tile.openstreetmap.org/){z}/{x}/{y}.png` (donde z es zoom, y x, y son las coordenadas del cuadrante).
 - **MarkerLayer (Capa de Marcadores)**: Es una capa transparente que se encabalga sobre el mapa. Permite posicionar cualquier Widget de Flutter (como un `Icon(Icons.location_on)`) en un punto geográfico exacto definido por un objeto **LatLng** (Latitud y Longitud).
 - **LatLng (Paquete latlong2)**: Una pequeña librería de matemáticas geospaciales complementaria que maneja las coordenadas en formato de punto flotante y permite calcular distancias en metros o líneas de rumbo entre dos puntos del planeta.
-

2. Configuración del Entorno (Plataforma Nativa)

Dado que este módulo reutiliza el hardware de geolocalización, requiere los mismos permisos del tutorial de GPS, sumando el acceso a Internet para la descarga de los mosaicos del mapa.

En Android (`android/app/src/main/AndroidManifest.xml`)

Asegúrate de que el permiso de Internet esté presente junto a los de ubicación fina antes de la etiqueta `<application>`:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

En iOS (`ios/Runner/Info.plist`)

Mantenemos la llave explicativa de geolocalización nativa:

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>Necesitamos acceso a tu ubicación para centrar el mapa digital en tu
posición actual.</string>
```

Estructura de Carpetas del Módulo

Añadimos la nueva característica (`feature`) llamada `maps` dentro del árbol del proyecto:

```
mi_app_sensores/
├── pubspec.yaml
├── lib/
│   ├── main.dart
│   ├── core/
│   │   └── theme/
│   │       └── theme.dart
│   └── features/
│       ├── home/
│       ├── gps/
│       └── maps/                                <-- CARPETA DEL MÓDULO ACTUAL
│           ├── screens/
│           └── map_screen.dart
```

Agreguen las siguientes dependencias en su `pubspec.yaml` (mantenemos `geolocator` porque el mapa necesita al GPS para saber dónde ubicarse):

```
dependencies:
  flutter:
    sdk: flutter
  geolocator: ^13.0.0
  flutter_map: ^7.0.2 # El motor del mapa Open Source
  latlong2: ^0.9.1    # Manejo de coordenadas matemáticas Lat/Lng
```

3. Código Fuente Completo y Comentado

Archivo `lib/features/maps/screens/map_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_map/flutter_map.dart';
import 'package:latlong2/latlong.dart';
import 'package:geolocator/geolocator.dart';
import 'dart:async';

class MapScreen extends StatefulWidget {
  const MapScreen({super.key});

  @override
  State<MapScreen> createState() => _MapScreenState();
}

class _MapScreenState extends State<MapScreen> {
  // Coordenadas iniciales por defecto (por ejemplo, Ciudad de México)
  LatLng _currentLatLng = const LatLng(19.4326, -99.1332);

  // Controlador para manipular la cámara del mapa (Zoom y Centro)
  final MapController _mapController = MapController();

  StreamSubscription<Position>? _gpsStreamSubscription;
  bool _isGpsReady = false;
  String _loadingText = "Iniciando mapa y buscando satélites...";

  @override
  void initState() {
    super.initState();
    _initMapWithGps();
  }

  // 1. Algoritmo de inicialización combinada: Permisos + Obtención de Coordenadas
  Future<void> _initMapWithGps() async {
    bool serviceEnabled;
    LocationPermission permission;

    // Verificar si el hardware de ubicación global está activo
    serviceEnabled = await Geolocator.isLocationServiceEnabled();
    if (!serviceEnabled) {
      setState(() { _loadingText = "Por favor, enciende el GPS del teléfono."; });
      return;
    }

    permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
      permission = await Geolocator.requestPermission();
      if (permission == LocationPermission.denied) {
        setState(() { _loadingText = "Permiso de ubicación denegado."; });
      }
    }
  }
}
```

```
        return;
      }
    }

    if (permission == LocationPermission.deniedForever) {
      setState(() { _loadingText = "Permisos bloqueados permanentemente en
Ajustes."; });
      return;
    }

    // Obtener la primera posición del satélite de forma síncrona para centrar el
mapa
    try {
      Position position = await Geolocator.getCurrentPosition(
        locationSettings: const LocationSettings(accuracy: LocationAccuracy.high)
      );

      setState(() {
        _currentLatLng = LatLng(position.latitude, position.longitude);
        _isGpsReady = true;
      });

      // Mover la cámara del mapa a las coordenadas reales con un zoom de 16
(nivel calle)
      _mapController.move(_currentLatLng, 16.0);

      // 2. Encender el Stream para actualizar el marcador reactivamente si el
usuario camina
      _startLiveTracking();

    } catch (e) {
      setState(() { _loadingText = "Error al conectar con el sensor GPS: $e"; });
    }
  }

  // 3. Monitoreo reactivo de movimiento para actualizar el marcador en el mapa
void _startLiveTracking() {
  const LocationSettings settings = LocationSettings(
    accuracy: LocationAccuracy.high,
    distanceFilter: 3, // Se actualiza si el alumno se mueve más de 3 metros
  );

  _gpsStreamSubscription = Geolocator.getPositionStream(locationSettings:
settings)
    .listen((Position position) {
      if (!mounted) return;

      setState(() {
        _currentLatLng = LatLng(position.latitude, position.longitude);
      });

      // Opcional: Centrar el mapa automáticamente en cada paso del usuario
      _mapController.move(_currentLatLng, _mapController.camera.zoom);
    });
}
```

```

}

@override
void dispose() {
  // CRUCIAL: Apagar el GPS y limpiar el controlador del mapa para evitar fugas
  de RAM
  _gpsStreamSubscription?.cancel();
  _mapController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Mapa Open Source & GPS')),
    body: Stack(
      children: [
        // Capa Base: Renderizador del mapa interactivo
        FlutterMap(
          mapController: _mapController,
          options: MapOptions(
            initialCenter: _currentLatLng,
            initialZoom: 13.0,
            maxZoom: 18.0,
            minZoom: 3.0,
          ),
          children: [
            // Capa 1: Descarga de mosaicos desde el servidor gratuito de
            OpenStreetMap
            TileLayer(
              urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
              userAgentPackageName: 'com.example.mi_app_sensores',
            ),

            // Capa 2: Marcador dinámico que sigue la posición del GPS
            if (_isGpsReady)
              MarkerLayer(
                markers: [
                  Marker(
                    point: _currentLatLng,
                    width: 50,
                    height: 50,
                    alignment: Alignment.topCenter, // Asegura que la punta del
                    pin apunte exacto a la coordenada
                    child: const Icon(
                      Icons.location_on,
                      color: Colors.red,
                      size: 45,
                    ),
                  ),
                ],
              ),
            ],
          ),
        ],
      ),
    ),
  ),

```

```

        // Capa de bloqueo superior (Pantalla de carga mientras el chip GPS
engancha los satélites)
        if (!_isGpsReady)
        Container(
            color: Colors.white,
            child: Center(
                child: Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                        const CircularProgressIndicator(),
                        const SizedBox(height: 20),
                        Text(
                            _loadingText,
                            textAlign: TextAlign.center,
                            style: const TextStyle(fontSize: 15, fontWeight:
FontWeight.bold),
                        ),
                    ],
                ),
            ),
        ),

        // Botón flotante para re-centrar el mapa manualmente en la ubicación
del alumno
        if (_isGpsReady)
        Positioned(
            bottom: 20,
            right: 20,
            child: FloatingActionButton(
                onPressed: () {
                    _mapController.move(_currentLatLng, 16.0);
                },
                backgroundColor: Colors.blueAccent,
                child: const Icon(Icons.my_location, color: Colors.white),
            ),
        ),
    ],
),
);
}
}

```

4. Guía Didáctica

- **El Mecanismo de Cache Gratuito:** A diferencia de Google Maps que cobra por cada petición cargada en pantalla, `flutter_map` descarga imágenes PNG estáticas transparentemente y las almacena en la memoria caché del teléfono automáticamente.
- **La Propiedad `userAgentPackageName`:** En la capa `TileLayer` incluimos `userAgentPackageName`. Los servidores de OpenStreetMap son gratuitos y sostenidos por la comunidad, por lo que exigen que las

aplicaciones se identifiquen con su id de paquete (User-Agent) para mitigar ataques de denegación de servicio (DDoS) o descargas masivas malintencionadas.

- **El Paradigma de Superposición (Stack):** La interfaz utiliza un widget `Stack`. Esto enseña a encimar capas de UI de control (como el indicador de carga o el botón flotante de "Mi ubicación") por encima de un lienzo gráfico pesado que ocupa toda la pantalla.

Vinculación en el Menú Principal

Abre tu archivo central `lib/features/home/screens/home_screen.dart` e integra el módulo sustituyendo o añadiendo la tarjeta correspondiente al mapa:

```
// 1. Agrega el import al inicio del archivo:
import '../maps/screens/map_screen.dart';

// 2. Modifica o añade la tarjeta dentro del GridView de home_screen.dart:
_SensorCard(
  title: 'Mapas Libres & GPS',
  icon: Icons.map,
  color: Colors.purple, // Asignamos color púrpura para este nuevo laboratorio
  onTap: () {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => const MapScreen()),
    );
  },
),
```